

Dynamic Memory Allocation: #include <stdlib.h>
 Allocation & Release → Runtime (Program Execution)

① malloc (size of data); Memory Allocation → whole block
 ② calloc (no of elements, size); Continuous/Contiguous Memory
 ③ realloc (ptr/memory, new size); Reallocation of Memory
 ④ free (ptr/memory); Free/Release of Memory

garbage/junk
 continuous block
 i → zeroes
 c → null
 b → false
 default values.

[malloc, calloc, realloc] → return type → void pointer.
 (int*) malloc() ← Integer (int*)
 (char*) malloc() ← Character (char*) → Type Casting

Q. Write a C Program to allocate & deallocate memory during program execution to represent an array of size 'n'.
 * Illustrate the use of all the functions:

① ✓ → malloc
 ② ✓ → calloc
 ✓ → realloc
 ✓ → free

Sample I/p: size → n = 5
 arr = 1, 2, 3, 4, 5
 new size → n = 8 → (realloc)
 arr = 1, 2, 3, 4, 5, 6, 7, 8

pointers

* Dangling Pointer?
 When a pointer points to a location which is:

- ① Erased/Deleted/Deallocated
- ② In another scope
- ③ Points to NULL

This scenario is called a dangling pointer & the pointer is called dangling pointer.

Traditional Swap
 temp
 temp = a
 a = b
 b = temp

call by value → copy
 function swap(a, b)
 a = 10 → 20
 b = 20 → 10

main's address
 a = 10
 b = 20 → copies
 swap(a, b)

12000 modify
 10
 3000
 20

Storage Classes: →

- ① Static scope global
- ② extern global
- ③ register RAM
- ④ auto → Local

int main() {
 x = 10;
 }

We cannot use pointers.

Lifetime
 Single File
 Multiple Files
 RAM Memory
 Single file
 Single block

f1.c
 static a
 extern b

f2.c
 a
 b

folder

block

C++
 auto

int* function() {
 int x = 20;
 return &x; (2000)
 }

main {
 int* ptr = function();
 printf("%d", *ptr);
 }

x [2000] → 20
 address/pointer
 2000

function →
 void max_min_using_pointers(int arr[], int n) {
 }

W → correct X
 solve by tomorrow

main() {
 int arr = [2, 0, -3, 6, 122, -5]
 max = 122
 min = -5
 print (mx, mn) ⇒ 122, -5
 }